Data-Driven Problem Solving

The Cooper Union for the Advancement of Science and Art

ME 371 - Fall 2025

Masoud Masoumi

Agenda

1. **Modeling Fundamentals**
2. **Classification**
    - What is classification?
    - Classification algorithms overview
3. **K-Nearest Neighbors (KNN)**
    - Algorithm overview and implementation
    - Distance metrics and choosing K
    - Evaluation metrics
    - Implementation in Python
4. **Logistic Regression**
    - From linear regression to classification
    - Sigmoid function and probability
    - Implementation in Python

Learning from Data & Modeling

Learning from Data

The basic premise of learning from data is using observations to uncover an underlying process.

Supervised Learning: Training data contains explicit examples of correct outputs for given inputs (input, output pairs). Used for prediction and classification.

Unsupervised Learning: Training data does not contain output information. Used for finding patterns and structure in data (e.g., clustering).

Reinforcement Learning: Learning through trial and error to maximize rewards. The algorithm discovers optimal actions through experimentation.

The Modeling Process

Modeling is the process of encapsulating information into a tool that can forecast and make predictions.

Key Concept: Split data into training, evaluation, and test sets

- **Training Data (60-70%):** Used to fit and learn the model parameters
- **Evaluation Data (15-20%):** Used to tune hyperparameters and compare different models
- **Test Data (15-20%):** Used for final evaluation on completely unseen data

Why three sets?

- Training set teaches the model
- Evaluation set helps you choose the best model without "peeking" at test data
- Test set gives honest assessment of real-world performance

General Procedure:

Data Preparation → Train/Eval/Test Split → Algorithm Setup → Model Fitting → Hyperparameter Tuning (using Eval) → Final Prediction → Evaluation (using Test)

Classification & Evaluation Metrics

What is Classification?

Classification is the problem of predicting the correct label for a given input record.

Key differences from regression:

- Labels are **discrete entities** (categories), not continuous values
- Target variable is **categorical**
- Supervised learning approach

Common Classification Algorithms:

- Decision Trees (ID3, C4.5, C5.0)
- Naive Bayes
- K-Nearest Neighbors
- Logistic Regression
- Support Vector Machines (SVM)
- Neural Networks

k-Nearest Neighbors (k-NN)

K-Nearest Neighbors (KNN)

Core Idea: Classify a point based on the labels of its nearest neighbors.

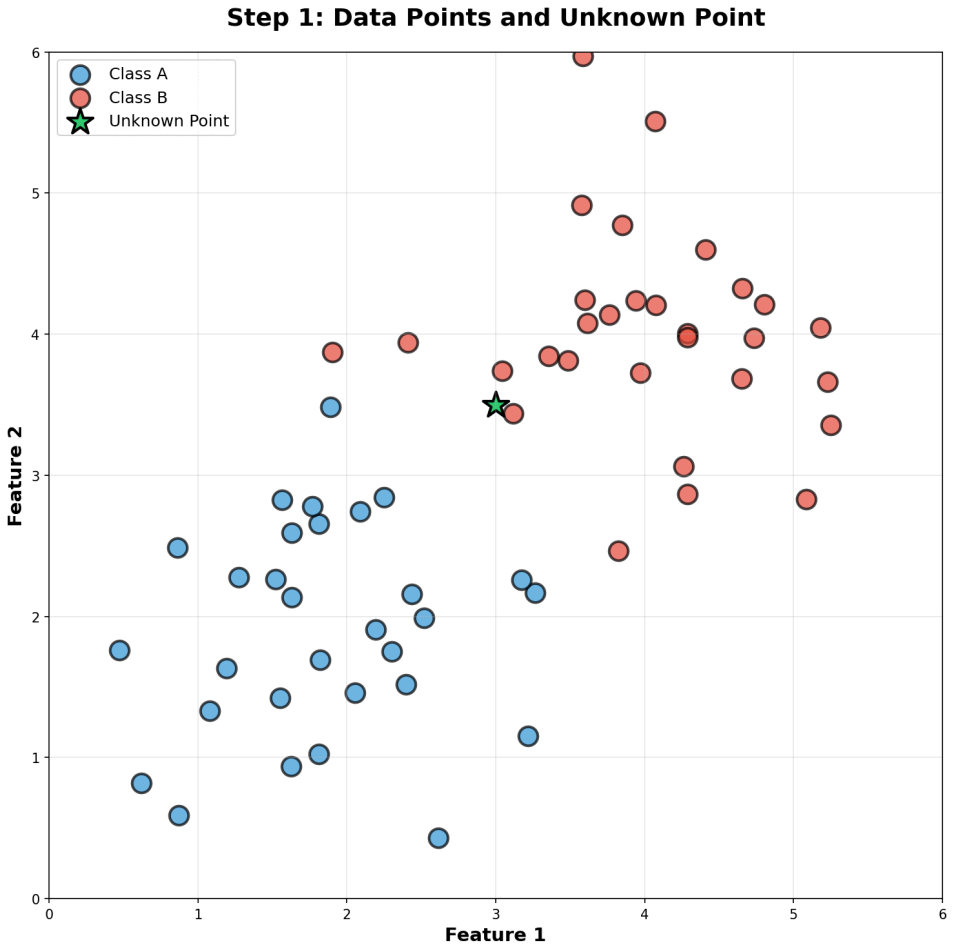
How it works:

1. Pick a value for K (number of neighbors)
2. Calculate distance from unknown point to all training points
3. Select the K nearest points
4. Predict using the most common class among K neighbors

Key Characteristics:

- Simple, intuitive algorithm
- No explicit training phase (lazy learning)
- Performance depends heavily on choice of K
- Sensitive to feature scaling

IN-CLASS ASSIGNMENT: Complete Part 1 of the in-class assignment.



KNN - Distance Metrics

Different distance metrics for measureing the distance between points $x^{(1)}$ and $x^{(2)}$ for all n features:

Euclidean Distance (most common)

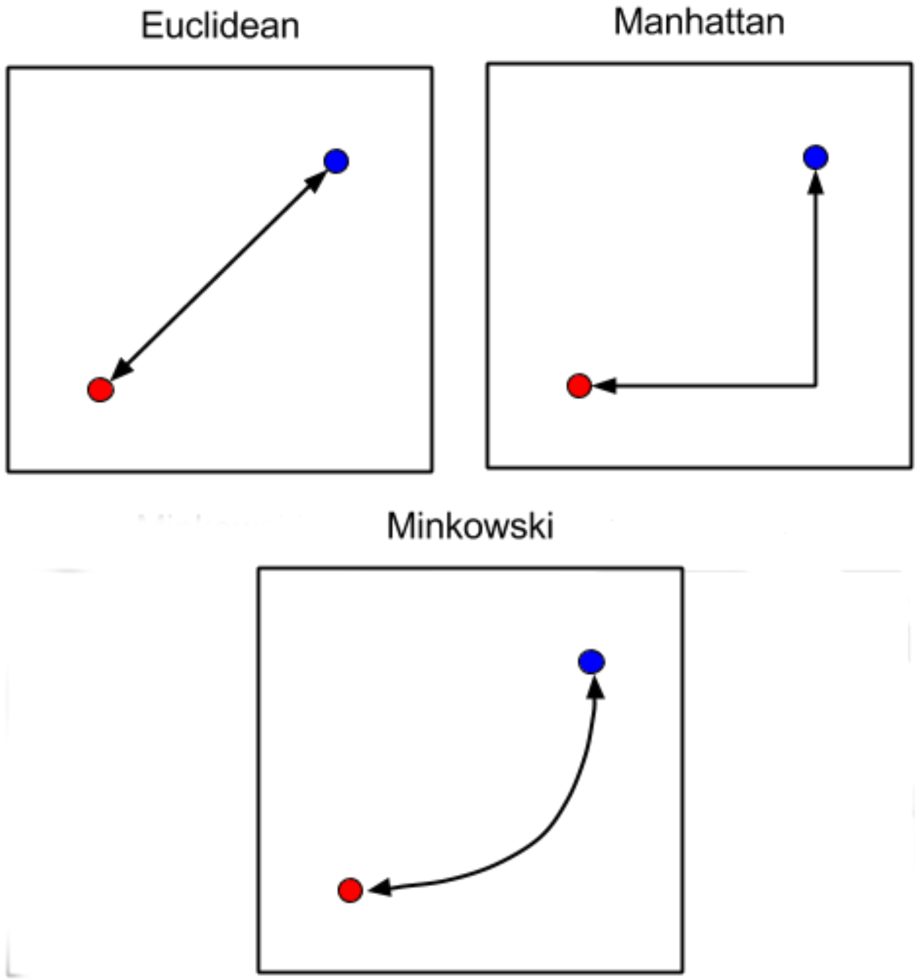
- d = $\sqrt{(\sum_{i=1}^{n}(x_i^{(1)} - x_i^{(2)})^2}$
- Straight-line distance

Manhattan Distance

- d = $\sum_{i=1}^{n} |x_i^{(1)} - x_i^{(2)}|$
- Sum of absolute differences

Minkowski Distance

- d = $(\sum_{i=1}^{n} |x_i^{(1)} - x_i^{(2)}|^p)^{\frac{1}{p}}$
- Generalization of Euclidean and Manhattan



Important: Always normalize/scale features before using KNN!

KNN in Python

Step 1: Import necessary libraries

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
```

Step 2: Prepare and normalize data

```
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3)
```

Step 3: Train and predict

```
knn = KNeighborsClassifier(n_neighbors=5)  # Choose K
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

IN-CLASS ASSIGNMENT: Complete Part 2 and Task 3.1 of the in-class assignment.

Model Evaluation

Evaluation Metrics - Confusion Matrix

Confusion Matrix - Shows all possible prediction outcomes:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Definitions:

- **True Positive (TP):** Correctly labeled positive
- **True Negative (TN):** Correctly labeled negative
- **False Positive (FP):** Incorrectly labeled positive (Type I error)
- **False Negative (FN):** Incorrectly labeled negative (Type II error)

Evaluation Metrics - Precision, Recall, F-Score, Accuracy

Precision = $\frac{TP}{(TP+FP)}$

- Measure of exactness
- What percentage of predicted positives are actually positive?

Recall = $\frac{TP}{(TP+FN)}$

- Measure of completeness
- What percentage of actual positives are correctly identified?

F-Score = $2 \times \frac{(Precision \times Recall)}{(Precision+Recall)}$

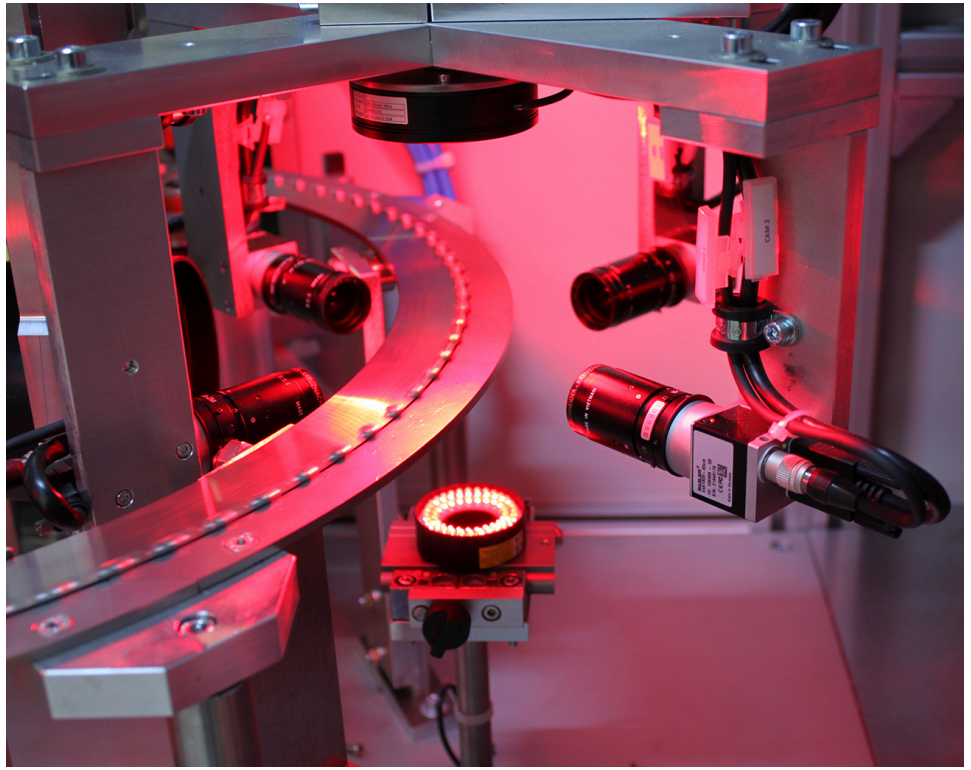
- Harmonic mean of precision and recall
- Balances both metrics

Accuracy = $\frac{(TP+TN)}{(TP+TN+FP+FN)}$

- Ratio of correct predictions over total predictions

Evaluation Metrics - Example

An automated quality control system in a quality control line separates **300 defected parts**, only **200 of which are actually defective (TP)**, **100 are non-defective (FP)** while failing to separate **400 additional defective parts (FN)**. In total, **1200 parts were inspected**. (source for image)



- Precision is 200/300 = 2/3 $\approx$ 66.7%
- Recall is 200/600 = 1/3 $\approx$ 33.3%
- Accuracy is $\frac{(200+500)}{(200+100+400+500)} = \frac{7}{12} \approx 58.3\%$
- F-Score is $2 \times \frac{0.667 \times 0.333}{0.667+0.333} \approx 44.4\%$

There tends to be an inverse relationship between precision and recall, where it is possible to increase one at the cost of reducing the other.

Model Evaluation - Implementation

```
from sklearn.metrics import accuracy_score, jaccard_score, f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
# Confusion matrix for binary classification
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=['Class 0', 'Class 1'])
disp.plot()
```

IN-CLASS ASSIGNMENT: Complete Part 3 of the in-class assignment.

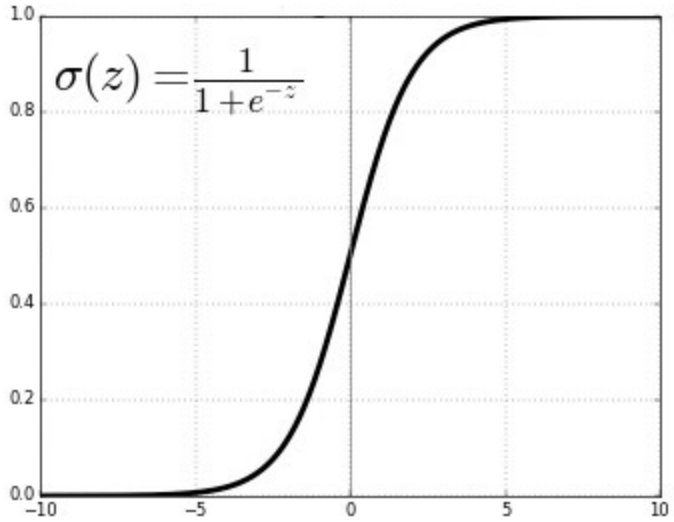
Logistic Regression

Logistic Regression Overview

The Challenge: Linear regression outputs continuous values, but we need probabilities between 0 and 1.

The Solution: Use the Sigmoid (Logistic) Function

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Key Properties:

- Input: any real number (-∞ to +∞)
- Output: probability between 0 and 1
- S-shaped curve
- Used to transform linear regression output into probability

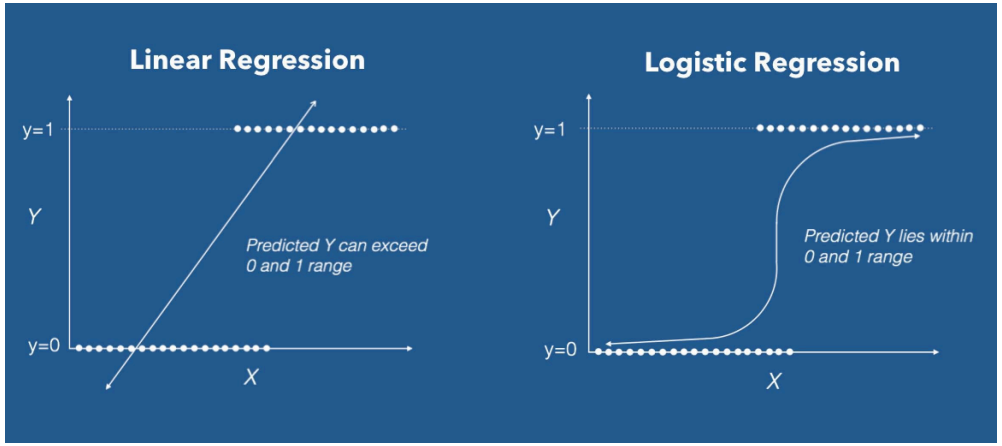
Logistic Regression - The Math

Linear Regression: $y = w^T x = w_0 + w_1 x_1 + w_2 x_2 + \ldots$

Logistic Regression: $P(y = 1|x) = \sigma(w^T x) = \frac{1}{1+e^{-w^T x}}$

Decision Making:

- If P(y=1 | x) ≥ 0.5, predict class 1
- If P(y=1 | x) < 0.5, predict class 0
- Threshold can be adjusted based on problem requirements



In statistical terms: We're finding the probability of the dependent variable being a specific class, given the selected features.

Logistic Regression in Python

```
# Step 1: Import necessary libraries
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# Step 2: Prepare data**
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3)

# Step 3: Train and predict**
lr = LogisticRegression(max_iter=500)
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
y_pred_proba = lr.predict_proba(X_test)  # Get probabilities
```

Logistic Regression - Log Loss

To evaluate our logistic regression model, we introduce a new metric for classifiers where the predicted output is a probability value between 0 and 1.

$$-\begin{cases} \log a & y = 1, \\ \log(1-a) & y = 0. \end{cases}$$

- For one prediction, we can define **Log Loss** as

$$-y \times log[\sigma(\mathrm{w}^T\mathrm{x})] - (1-y) \times log[1-\sigma(\mathrm{w}^T\mathrm{x})]$$

- For all m samples

Log Loss= $\frac{1}{m}\sum_{i=1}^{m} -y^{(i)} \times log[\sigma(\mathrm{w}^T\mathrm{x}^{(i)})] - (1-y^{(i)}) \times log[1-\sigma(\mathrm{w}^T\mathrm{x}^{(i)})]$

IN-CLASS ASSIGNMENT: Complete Part 4 of the in-class assignment.

Model Selection

Comparing KNN vs. Logistic Regression

K-Nearest Neighbors:

- ✓ Simple, intuitive
- ✓ No training time
- ✓ Works well with non-linear boundaries
- ✗ Slow prediction time for large datasets
- ✗ Sensitive to feature scaling
- ✗ Requires choosing K

Logistic Regression:

- ✓ Fast prediction time
- ✓ Provides probability estimates
- ✓ Works well with linearly separable data
- ✓ Less memory intensive
- ✗ Assumes linear decision boundary
- ✗ May require feature engineering

Summary

Modeling Process:

- Always split data into training and test sets
- Normalize/scale features for better performance
- Evaluate on multiple metrics, not just accuracy

Classification Algorithms:

- KNN: Simple, non-parametric, distance-based
- Logistic Regression: Probabilistic, linear decision boundary

Evaluation:

- Use confusion matrix to understand errors
- Precision, Recall, F1-score provide nuanced performance view
- Choose metrics based on problem requirements

IN-CLASS ASSIGNMENT: Complete Part 5 of the in-class assignment.